

RUAHTONE

Modelo de Dominio y Arquitectura

Entidades, relaciones, fronteras de datos y tecnología oficial

PROPÓSITO

Fuente profesional para diseñar, construir y validar el MVP frontend local de RUAHTONE.

RHT-ARCH-001 • Versión 1.0

12 de agosto de 2026

Estado: Documento oficial del MVP

CONTROL DE FUENTE

Este documento reorganiza exclusivamente la Instrucción Principal RUAHTONE — Master Product & Development Prompt. No agrega funcionalidades al alcance oficial.

1. Arquitectura conceptual

Los datos se modelan relacionalmente aunque el MVP no utilice backend. Las relaciones guardan identificadores y evitan duplicar información innecesaria.

Entidad	Responsabilidad oficial
Church	Organización principal; futura relación con plan y usuarios
User	Identidad de acceso conceptual, separada de permisos y ministerios
Person	Miembro o participante con rol, ministerios, disponibilidad y estado
Ministry	Área de servicio con líder, miembros, checklist y configuraciones
Service	Unidad operacional central
ServiceAssignment	Persona asignada a función y ministerio en un servicio
Song	Información global de biblioteca y tonalidad original
ServiceSong	Configuración contextual de una canción en un servicio
Resource	Recurso relacionado con canción, servicio o ministerio
Checklist / Item	Preparación contextual y estado de tareas
Activity	Historial estructurado de cambios
Notification	Aviso relevante para el usuario
SoundConfiguration	Preparación técnica de sala, monitoreo, P16, patch y escena

2. Relaciones principales

- 1. Una Iglesia organiza Ministerios, Personas y Servicios.
- 2. Una Persona puede pertenecer a varios Ministerios.
- 3. Un Servicio contiene Asignaciones que referencian Persona y Ministerio.
- 4. Un Servicio contiene ServiceSongs que referencian una Song global.
- 5. Recursos se relacionan con Canciones, Servicios o Ministerios.
- 6. Checklists se contextualizan por Servicio, Ministerio o Persona.
- 7. La Actividad registra modificaciones relevantes del Servicio.

3. Diferencia entre datos globales y contextuales

Dato global	Dato contextual
Song.originalKey = D	ServiceSong.key = F para Culto Domingo

Dato global	Dato contextual
Person y sus ministerios	ServiceAssignment y función en un servicio
Recurso de una canción	Recurso asociado a un servicio específico
Plantilla de checklist	Checklist ejecutado para un servicio

INVARIANTE CRÍTICA

Cambiar la tonalidad de una canción dentro de un servicio modifica únicamente ServiceSong. Song.originalKey permanece intacta.

4. Contratos mínimos oficiales

Entidad	Campos mínimos definidos
Person	id, nombre, apellido, avatar, teléfono opcional, ministerios, rol, disponibilidad, estado
ServiceAssignment	id, serviceId, personId, ministryId, función, attendanceStatus, notas
Song	id, nombre, autor, tonalidad original, BPM, duración, compás, etiquetas, recursos, historial
ServiceSong	id, serviceId, songId, orden, key, BPM opcional, versión, notas, enabled
Service	nombre, fecha, horas, lugar, descripción, ministerios, personas, canciones, recursos, checklist, actividad
Ministry	id, nombre, descripción, icono, color, líder, miembros, checklist, configuraciones

5. Reglas de integridad

- Las relaciones almacenan IDs, no nombres duplicados.
- Las estadísticas se calculan desde asignaciones y estados reales.
- Una confirmación cambia únicamente la asignación de la persona que responde.
- Una persona puede tener varios ministerios, pero una sola identidad.
- Los cambios importantes generan actividad.
- Los servicios históricos conservan canciones, tonalidades y asignaciones utilizadas.
- Un conflicto de horario se informa claramente.
- Los permisos se simulan aunque no exista autenticación real.

6. Estados oficiales

Entidad	Estados
Servicio	Planificación, Organizando, Confirmando, Listo, Completado, Archivado
Asistencia	Pendiente, Confirmado, No asistiré, Llegaré tarde
Prueba de sonido	Pendiente, En progreso, Completada

Entidad	Estados
Checklist	Ítems completos o pendientes

7. Tecnología oficial del MVP

- React, TypeScript y Vite.
- Tailwind CSS y shadcn/ui.
- Lucide Icons y Framer Motion.
- LocalStorage como persistencia local.
- React Context o Zustand solamente si simplifica el estado.
- Tipos e interfaces TypeScript para el dominio.

8. Restricciones técnicas

- No utilizar Firebase, Supabase, PostgreSQL, MongoDB ni APIs externas.
- No implementar backend, autenticación real ni servicios cloud.
- Crear un store central para personas, ministerios, servicios, asignaciones, canciones, recursos, checklists, actividad y configuración.
- Persistir cambios en LocalStorage y permitir restablecer el seed demo.
- Separar conceptualmente usuario, permisos, rol, ministerios y datos.

9. Preparación para evolución futura

Sin implementar estas capacidades, el diseño no debe bloquear aplicación móvil, autenticación, múltiples iglesias, multi-sede, pagos, notificaciones, almacenamiento cloud, roles personalizados, estadísticas, IA e integraciones.

- La futura estructura SaaS contempla Iglesia → Plan → Usuarios → Ministerios → Servicios.
- La lógica conceptual debe poder reutilizarse en una experiencia móvil.
- Sonido contempla streaming como área futura, sin activarlo en el MVP.

10. Estructura lógica sugerida por la especificación

Esta vista organiza las prioridades oficiales; no introduce una dependencia adicional.

- Capa de dominio: entidades, relaciones, estados y reglas.
- Store central: acciones y persistencia LocalStorage.
- Experiencias por contexto: servidor, líder, administrador, pastor, sonido y multimedia.
- Features operacionales: servicios, asignaciones, canciones, recursos, checklists, actividad, calendario y sonido.
- Datos demo: Iglesia Manantiales y escenarios consistentes.